

PIZZAGO

Online Pizza Ordering System

Alejandro, Tomas, Artur and Bradan

Academic Year: 2025 / 2026

Table of Contents

1. Introduction & System Description.....	2
2. Requirement Specification	3
2.1 Functional Requirements.....	3
2.2 Non-Functional Requirements	3
2.3 System Requirements.....	4
3. Use Case Diagram.....	4
3.1 Use Cases per Actor	4
3.2 Detailed Use Case Descriptions.....	4
4. Class Diagram	5
4.1 Package: models	5
4.2 Package: models.users	5
4.3 Package: models.ingredients.....	6
4.4 Package: models.pizzas	6
4.5 Package: models.factories.....	6
4.6 Package: services	6
4.7 Class Relationships	6
5. Screen / UI Design	7
5.1 Login / Register Screen	7
5.2 Customer Menu	7
5.3 Order Flow	7
5.4 Invoice	7
5.5 Admin Menu	7
6. Expected Milestone Schedule	7
7. Weekly Milestone Deliveries	8
8. Results Obtained	9
9. Problems Encountered.....	9
10. Future Improvements	9

1. Introduction & System Description

PizzaGO is a Java console application for managing pizza orders, developed as the final project for the Programming module at IES San Vicente. The system allows customers to browse the menu, place customized orders, and receive an invoice, while administrators have tools to manage the pizza catalogue, view all orders, and administer user accounts.

The problem it solves is the digitalization of the ordering process in a pizzeria, eliminating manual paper-based management. The primary objectives are:

- Provide customers with a guided, intuitive ordering process via the console.
- Allow pizza customization by adding extra ingredients to any pizza.
- Handle user authentication securely using BCrypt password hashing.
- Persist orders and user accounts in text files for later retrieval.
- Give the administrator tools to manage the menu and review all orders.

2. Requirement Specification

2.1 Functional Requirements

ID	Name	Description
FR-01	User registration	The system allows for creating a new account with a username and password.
FR-02	Login	The system authenticates the user by verifying credentials against login.txt using BCrypt.
FR-03	View menu	The customer can browse all available pizzas with their ingredients, price, and estimated time.
FR-04	Place order	The customer can select one or more pizzas and specify the quantity for each.
FR-05	Customize pizza	The customer can add extra ingredients to each pizza in the order.
FR-06	Save order	The system persists in the orders.txt with id, client, date, pizzas and total price.
FR-07	Print invoice	The system displays an order summary with the total amount at checkout.
FR-08	Admin - manage pizzas	The administrator can add or remove pizzas from the menu.
FR-09	Admin - view orders	The administrator can view all registered orders.
FR-10	Admin - view users	The administrator can list all registered users.

2.2 Non-Functional Requirements

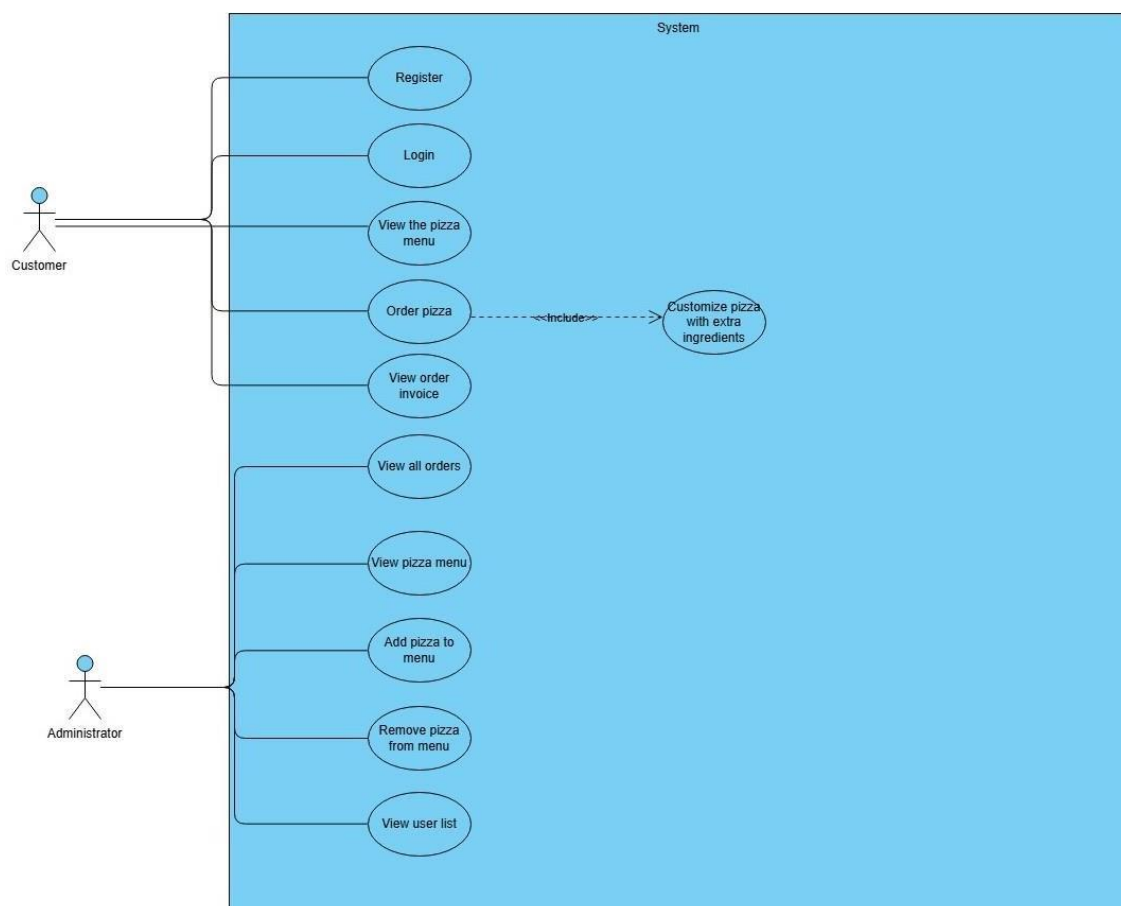
ID	Category	Description
NFR-01	Security	Passwords are stored hashed with BCrypt via the Jbcrypt library; plain-text passwords are never written to disk.
NFR-02	Maintainability	All public classes and methods are documented with JavaDoc; generated output is in the /doc folder.
NFR-03	Reliability	JUnit5 unit tests validate price and time calculations in OrderTest.
NFR-04	Portability	The application runs on any system with JDK 11+ from IntelliJ IDEA or the command line.
NFR-05	Persistence	Data is stored in plain .txt files (login.txt and orders.txt) with no database dependency.

2.3 System Requirements

- Java Development Kit (JDK) 11 or higher.
- IntelliJ IDEA (recommended) or any IDE supporting standard Java projects.
- Jbcrypt library for password hashing (included as project dependency).
- JUnit 5 library for running unit tests.
- File system with written permissions on the src/data/ directory.

3. Use Case Diagram

The system has two main actors: the Customer (a registered non-admin user) and the Administrator.



3.1 Use Cases per Actor

Actor: Customer

- UC-01: Register in the system
- UC-02: Log in
- UC-03: View the pizza menu
- UC-04: Place an order (includes UC-05)
 - UC-05: Customize pizza with extra ingredients
- UC-06: View order invoice

Actor: Administrator

- UC-02: Log in (shared with Customer)
- UC-07: View all orders
- UC-08: View pizza menu
- UC-09: Add pizza to menu
- UC-10: Remove pizza from menu
- UC-11: View user list

3.2 Detailed Use Case Descriptions

UC-04: Place an Order

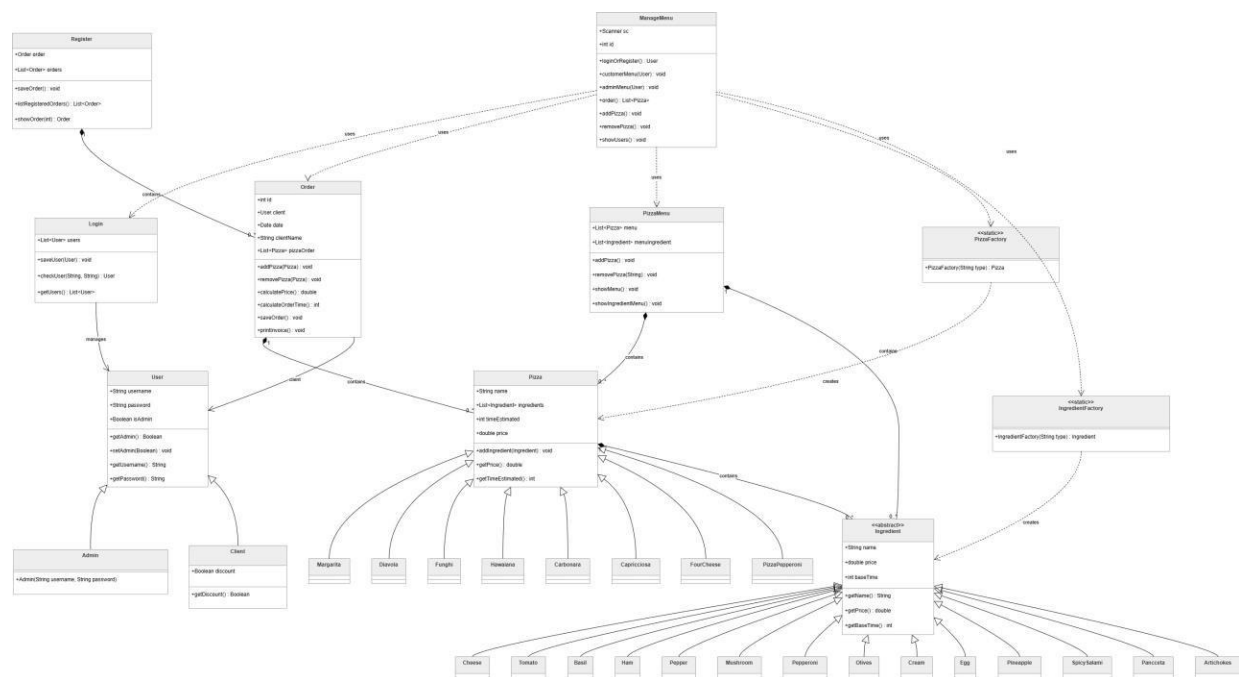
Primary actor	Authenticated Customer
Precondition	The user has successfully logged in.
Main flow	1. Customer selects 'Make an order'. 2. Enter the name of the desired pizza. 3. Enter the quantity. 4. Optionally add extra ingredients to each pizza. 5. Decides whether to add more pizzas. 6. System saves the order and prints the invoice.
Alternative flow	If the pizza name does not exist, the system shows the menu and returns to step 2.
Postcondition	The order is saved in orders.txt and the invoice is displayed.

UC-09: Add Pizza to Menu

Primary actor	Administrator
Precondition	The administrator has logged in.
Main flow	1. Selects 'Add pizza' from the admin menu. 2. Enter the name of the new pizza. 3. Enter how many ingredients to add. 4. For each ingredient, enter its name. 5. System validates and adds the pizza to the menu.
Alternative flow	If an ingredient does not exist, the system prompts another one.
Postcondition	The new pizza is available in the menu.

4. Class Diagram

The following describes the class architecture of PizzaGO, organized by package.



4.1 Package: models

Class	Key Attributes	Methods / Notes
Pizza	name: String ingredients: List<Ingredient> timeEstimated: int price: double	addIngredient(Ingredient) getPrice(), getTimeEstimated() toString(), equals(), hashCode()
Ingredient (abstract)	name: String price: double baseTime: int	Abstract base class for all ingredients. Getters and setters for all fields.
PizzaMenu	menu: List<Pizza> menuIngredient: List<Ingredient>	addPizza(), removePizza(String) showMenu(), showIngredientMenu()
User	username: String password: String isAdmin: Boolean	getAdmin(), setAdmin(Boolean) getUsername(), getPassword()

4.2 Package: models.users

Class	Inherits from	Description
Admin	User	Extends User. Constructor calls setAdmin(true). Represents an administrator account.
Client	User	Extends User. Adds discount: Boolean attribute and getDiscount() method.

4.3 Package: models.ingredients

Concrete classes extending Ingredient: Artichokes, Basil, Cheese, Cream, Egg, Ham, Mushroom, Olives, Pancetta, Pepper, Pepperoni, Pineapple, SpicySalami, Tomato. Each calls super(name, price) with its specific values.

4.4 Package: models.pizzas

Concrete classes extending to Pizza: Capricciosa, Carbonara, Diavola, FourCheese, Funghi, Hawaiana, Margarita, PizzaPepperoni. Each calls super(name) and adds its ingredients via addIngredient().

Example (Margarita): addIngredient(new Cheese()), addIngredient(new Tomato()), addIngredient(new Basil()).

4.5 Package: models.factories

Class	Description
PizzaFactory	Factory pattern. Static method PizzaFactory(String type) returns the matching Pizza instance by name. Returns null if not found.
IngredientFactory	Factory pattern. Static method IngredientFactory(String type) returns the matching Ingredient instance by name. Returns null if not found.

4.6 Package: services

Class	Associations	Responsibility
Login	Uses User, Admin, Client	Loads and saves users in login.txt. Verifies credentials with BCrypt.
Order	Uses User, Pizza, Client	Represents an order. Calculates price and time. Saves to orders.txt. Prints invoice.
Register	Contains Order, List<Order>	In-memory order history. saveOrder(), listRegisteredOrders(), showOrder(int id).
ManageMenu	Orchestrates all services	Manages console menus (loginOrRegister, customerMenu, adminMenu) and pizza CRUD operations.

4.7 Class Relationships

- Inheritance: Admin extends User, Client extends User.
- Inheritance: all ingredient classes extend the abstract Ingredient.
- Inheritance: all concrete pizza classes extend Pizza.
- Composition: Pizza contains List<Ingredient> (1 pizza has 0..* ingredients).
- Association: PizzaMenu contains List<Pizza> and List<Ingredient>.
- Association: Order contains User (client) and List<Pizza>.
- Association: Register contains Order and List<Order>.
- Dependency: ManageMenu uses Login, PizzaMenu, Order, PizzaFactory, IngredientFactory.

5. Screen / UI Design

The application has a console-based interface (CLI). The main screen flows are described below.

5.1 Login / Register Screen

```
===== ADMIN MENU =====  
1. Show orders  
2. Show pizzas  
3. Add pizza  
4. Remove pizza  
5. Show users  
0. Exit  
Choose option: 5  
Admin1 - Admin  
Admin2 - Admin  
Ale - Client  
Ale2 - Client
```

5.2 Customer Menu

Choose an option:

- 1.- Login
- 2.- Register
- 0.- Exit

1

Enter username: *Ale*

Enter password: *1234*

Welcome to PizzaGO

What do you want to do?

- 1.- Show pizza menu.
- 2.- Make an order.
- 0.- Exit.

1

Pizza Capricciosa, ingredients: 6, time: 16 minutes, price: 16.98€

Pizza Carbonara, ingredients: 4, time: 14 minutes, price: 13.48€

Pizza Diavola, ingredients: 3, time: 13 minutes, price: 11.25€

Pizza FourCheese, ingredients: 5, time: 15 minutes, price: 13.5€

Pizza Funghi, ingredients: 3, time: 13 minutes, price: 11.74€

Pizza Hawaiana, ingredients: 4, time: 14 minutes, price: 14.73€

Pizza Margarita, ingredients: 3, time: 13 minutes, price: 10.74€

Pizza Pepperoni, ingredients: 3, time: 13 minutes, price: 10.9€

5.3 Order Flow

```
↑ What do you want to do?
↓ 1.- Show pizza menu.
⇅ 2.- Make an order.
⇅ 0.- Exit.
⇅ 2
⇅ What pizza would you like to order?: Capricciosa
⇅ How many pizzas of this type would you like to order?: 1
⇅ Do you want to add ingredients to pizza 1? [y/n]: y
⇅ What ingredient do you want to add?
⇅ Artichokes:2.25€
⇅ Basil:0.99€
⇅ Cheese:1.25€
⇅ Cream:1.99€
⇅ Egg:0.75€
⇅ Ham:1.99€
⇅ Mushroom:1.99€
⇅ Olives:1.0€
⇅ Pepperoni:1.15€
⇅ Pancetta:1.49€
⇅ Pepper:0.88€
⇅ Pineapple:2.99€
⇅ SpicySalami:1.5€
```

5.4 Invoice

```
SpicySalami:1.5€
Tomato:0.5€
Ingredient name: Tomato
Add another ingredient? [y/n]: n
Do you want to order another pizza? [y/n]: n
```

Order saved successfully.

🍕 P I Z Z A G O 🍕

```
Order ID : #2
Client   : Ale
Date     : Thu May 28 22:19:33 CEST 2026
Capricciosa          17,48€
AMOUNT               17,48€
Est. time: 17.0 min
;Thanks for your order!
```

What do you want to do?

- 1.- Show pizza menu.
- 2.- Make an order.
- 0.- Exit.

5.5 Admin Menu

Choose an option:

- 1.- Login
- 2.- Register
- 0.- Exit

1

Enter username: Admin1

Enter password: 1234

===== ADMIN MENU =====

- 1. Show orders
- 2. Show pizzas
- 3. Add pizza
- 4. Remove pizza
- 5. Show users
- 0. Exit

Choose option: 1

Id: 1, Name: Ale, Date: Thu May 28 22:12:52 CEST 2026, Client: Ale, PizzaOrder: pizzilla1, time:10 min, price:8.0 €

Id: 2, Name: Ale, Date: Thu May 28 22:19:33 CEST 2026, Client: Ale, PizzaOrder: Capricciosa, time:16 min, price:16.98 €

===== ADMIN MENU =====

- 1. Show orders
- 2. Show pizzas

6. Expected Milestone Schedule

Week	Phase	Planned Tasks
Week 1	Stage 0 - Definition & Initial Analysis	Choose the system, write the project description, and define functional and non-functional requirements.
Weeks 2	Stage 1 - Analysis & UML Design	Complete system requirements, create use case diagram and class diagram.
Weeks 3	Stage 1 - UI & Initial Implementation	Design screen mockups and application flow. Implement Ingredient hierarchy and Pizza classes.
Week 4	Stage 2 - Core Implementation	Implement User hierarchy (User, Admin, Client), Login with BCrypt, PizzaFactory and IngredientFactory.
Weeks 5	Stage 2 - Services & Persistence	Implement Order and ManageMenu classes. Add File I/O for login.txt and orders.txt.
Weeks 6	Stage 2 - Testing & Final Delivery	Implement JUnit tests, generate JavaDoc, integration testing, bug fixing, update Memoria and final submission.

7. Weekly Milestone Deliveries

Week	Work Completed
Wk 1	Project definition completed: pizza ordering system with customer and administrator roles. Functional and non-functional requirements are written.
Wk 2	System requirements are finalized. Use case diagram and first draft of class diagram completed.
Wk 3	Designed UI mockups and application flow. Implemented abstract class Ingredient and all ingredient subclasses. Implemented Pizza class and concrete pizza subclasses.
Wk 4	Implemented user hierarchy: User, Admin, Client. Integrated Jbcrypt for password hashing. Implemented Login with BCrypt. Created PizzaFactory and IngredientFactory.
Wk 5	Implemented Order functionalities: addPizza, removePizza, calculatePrice, calculateOrderTime, saveOrder, printInvoice. Implemented ManageMenu with customerMenu, adminMenu and loginOrRegister. Added persistence with File I/O.
Wk 6	Designed and implemented OrderTest with JUnit 5. Generated JavaDoc. Performed integration testing, bug fixes, updated Memoria and prepared final submission.

8. Results Obtained

Technical Requirement	Status	Details
OOP: classes, encapsulation, polymorphism	Met	Private fields with getters/setters. Polymorphism through Ingredient and Pizza hierarchies.
Inheritance (minimum 1)	Met	Admin extends User, Client extends User, Margarita extends Pizza, Cheese extends Ingredient, etc.
Class association	Met	Pizza-Ingredient (1:N), Order-Pizza (1:N), Order-User (1:1).
Collections	Met	List<Pizza> in PizzaMenu and Order. List<Ingredient> in Pizza. List<User> in Login.
File I/O	Met	Login: read/write login.txt. Order: Write to orders.txt; read in showOrders().
JUnit testing	Met	OrderTest with @BeforeEach and @Test for calculatePrice and calculateOrderTime.
JavaDoc documentation	Met	Documentation generated in /doc folder covering all classes and packages.

Fully functional features: user registration and login, menu browsing, order placement and customization, file persistence, invoice printing, and the full administration panel.

9. Problems Encountered

Problem	Solution Applied
addPizza() in PizzaMenu initialises 'pizza' as null before the ingredient loop, causing NullPointerException.	Instantiate Pizza with the captured name before the ingredient loop: pizza = new Pizza(name).
removePizza() calls menu.remove() inside a for-each loop, throwing ConcurrentModificationException.	Use Iterator.remove() or collect the target element first, then remove it after the loop.
PizzaFactory maps 'pepper' to Carbonara instead of the correct pizza.	Change the case to 'carbonara' so it correctly returns new Carbonara().
Admin detection in Login.checkUser() fails due to operator precedence in the OR condition.	Wrap username checks in parentheses: (Admin1 Admin2) && BCrypt.checkpw(...).
getDiscount() in Client always returns true, ignoring the instance field.	Change the method to return this discount instead of the hard-coded literal.
showUsers() creates an empty Login instance, so getUsers() always returns an empty list.	Load users from the file before iterating by calling the file-reading logic first.

10. Future Improvements

- Replace plain-text file persistence with a SQLite or H2 embedded database.
 - Develop a graphical user interface (GUI) using JavaFX or Swing.
 - Implement a real discount/loyalty system for clients.
 - Allow customers to edit their order before confirming (remove pizzas from the cart).
 - Add password validation rules (minimum length, special characters).
 - Extend unit test coverage: Login, PizzaMenu, and factory classes.
 - Implement a flexible role system instead of hard-coded admin usernames.
 - Add internationalization (i18n) support for multiple languages.
 - Add per-customer order history so clients can review past orders.
-